Agentic Sales Orchestrator

Agent Run Log Programmatic Table Creation Runbook

Enterprise implementation guide with true line-by-line C# script explanation

Attribute	Value
Environment	Phoenicarix-CI
Dataverse URL	https://phoenicarix-ci.crm4.dynamics.com
Solution	ASO.Core
Solution unique name	ASOCore
Table	Agent Run Log
Logical name	aso_agentrunlog
Run method	macOS Terminal / .NET 8 / Dataverse Client SDK
Version	v1.0
Date	2026-05-24

Executive summary

We created a new custom Dataverse table called Agent Run Log in ASO.Core. The table provides a governed audit and traceability store for AI agents, Foundry orchestration, Customer Insights journeys, future SAP wrapper calls, and other Agentic Sales Orchestrator components. In plain language: every important automation or AI run gets a receipt that customer IT can inspect later.

Business objective

The objective is to make AI and orchestration actions transparent, diagnosable, and auditable. The table does not perform automation by itself. It prepares the place where future automation can write what happened, why it happened, when it happened, and how to trace the run.

Technical objective

Create the organization-owned custom table aso_agentrunlog, create the primary name column aso_name, create run-log metadata columns, reuse global choices where defined, publish the table, and keep all created metadata inside the ASO.Core solution layer.

Scope

The script creates the Agent Run Log table and columns only. It does not create flows, Foundry agents, SAP calls, Customer Insights journeys, forms, views, or records.

Out of scope

No customer communication, no SAP integration activation, no AI execution, no journey activation, no data migration, and no security model finalization are included in this step.

Audience-oriented explanation

Audience	Explanation
Grandma / non-technical stakeholder	We created a logbook. When a digital assistant later does something, it can write down what it did and whether it worked.
Product Owner	This creates the auditability foundation for AI-assisted sales orchestration. It supports traceability, operational review, and acceptance criteria for safe automation.
Junior developer	This is a .NET script that connects to Dataverse, checks if the custom table exists, creates it if missing, creates columns if missing, and publishes the table.
Senior technical consultant	This is a metadata deployment pattern using ServiceClient, CreateEntityRequest, CreateAttributeRequest, RetrieveEntityRequest, global option set binding, and targeted PublishXmlRequest with SolutionUniqueName = ASOCore.

Prerequisites

- Correct environment selected: Phoenicarix-CI.
- ASO.Core exists and has unique name ASOCore.
- Publisher prefix aso exists.
- Global choices aso_agenttype and aso_aistatus exist.
- .NET 8 SDK is installed on macOS.
- Microsoft.PowerPlatform.Dataverse.Client package can be restored.
- User has permissions to create tables, create columns, and publish customizations.
- Managed and unmanaged ASO.Core backups exist before running schema changes.

Architecture and context

The Agent Run Log table belongs to the operational observability layer of the Agentic Sales Orchestrator. It is not the seller workspace itself. It supports later flows and agents by providing a Dataverse-native audit trail for orchestration runs. The business tables such as Lead, Opportunity, Account, and Contact store current business state. Agent Run Log stores execution history and technical traceability.

Field inventory created on Agent Run Log

Display name	Schema name	Type	Values / size	Business purpose
Agent Type	aso_agenttype	Global choice	aso_AgentType	Identifies which agent, child agent, journey, or integration component produced the log entry.
Record Type	aso_recordtype	Text	100	Stores the related business record type such as lead, opportunity, account, contact, or a custom table name.
Record ID	aso_recordid	Text	100	Stores the related record identifier without forcing a polymorphic lookup in this MVP.
Message ID	aso_messageid	Text	100	Stores a message/event identifier for replay and traceability.
Correlation ID	aso_correlationid	Text	100	Connects Power Automate, Foundry, Dataverse, future APIM/Functions, and monitoring traces.
Input Payload	aso_inputpayload	Multiline text	4000	Captures the request/context sent into the agent or orchestration run.
Output Payload	aso_outputpayload	Multiline text	4000	Captures the response or decision returned by the agent or orchestration run.
Confidence	aso_confidence	Decimal	0.00-1.00 precision 2	Stores a normalized confidence score to drive human review or escalation rules.
Status	aso_status	Global choice	aso_AIStatus	Tracks execution status using the shared ASO AI status taxonomy.
Error Message	aso_errormessage	Multiline text	4000	Stores error details if the run failed.
Started On	aso_startedon	Date and time	User local	Captures when execution started.
Finished On	aso_finishedon	Date and time	User local	Captures when execution completed.
Trace ID	aso_traceid	Text	100	Stores observability trace references for diagnostics.

Sources Used	aso_sourcesused	Multiline text	4000	Stores sources, systems, and context used by the run.
SAP Called	aso_sapcalled	Yes/No	Default No	Indicates whether SAP was called through the governed path.
Retry Count	aso_retrycount	Whole number	0-100	Stores retry attempts for operational diagnosis.
Foundry Run ID	aso_foundryrunid	Text	100	Stores Microsoft Foundry run identifier.
Sales Agent Run ID	aso_salesagentrunid	Text	100	Stores Dynamics Sales Agent/Copilot run identifier.
Customer Insights Journey ID	aso_customerinsightsjourneyid	Text	100	Stores related Customer Insights journey identifier if the log belongs to a journey action.

Global choices referenced

Global choice	Used by	Values / purpose	Notes
aso_agenttype	Agent Type	FoundryParent, FoundryChildLeadOrigination, FoundryChildNurturing, FoundryChildOpportunityClassification, FoundryChildRiskCompetitor, FoundryChildNextBestAction, FoundryHandoff, SalesQualificationAgent, SalesOpportunityAgent, SalesDealCloseAgent, CustomerInsightsJourney, SAPWrapper	Identifies the source of the run log.
aso_aistatus	Status	NotStarted, Running, Completed, Escalated, Failed, PartiallyCompleted, Blocked	Keeps status values consistent across AI and orchestration-related fields.

Step-by-step implementation summary

- Created folder ~/aso-dataverse-tools/agent-run-log.
- Created a .NET 8 console application.
- Added Microsoft.PowerPlatform.Dataverse.Client.
- Pasted the Program.cs script.
- Ran dotnet run.
- Script authenticated to Phoenicarix-CI.
- Script checked if aso_agentrunlog exists.
- Script created the custom table because it did not exist.
- Script created the missing columns.
- Script published the table customizations.
- Result was validated in ASO.Core -> Tables -> Agent Run Log -> Columns.
- ASO.Core was backed up using the agreed managed/unmanaged naming pattern.

Validation checklist

Check	How to validate	Expected result

Table exists	ASO.Core -> Tables	Agent Run Log is visible
Logical name	Open table properties	aso_agentrunlog
Columns exist	Agent Run Log -> Columns	Run-log aso_ fields are visible
Global choices	Open Agent Type and Status columns	They reference existing global choices
Terminal success	Review terminal output	Green CREATED lines and Done message
Backup	Check exported files	Managed and unmanaged backups exist

Troubleshooting guide

Symptom	Likely cause	Action
Connection failed	Wrong environment URL or authentication issue	Verify Phoenicarix-CI URL and sign in again.
Global choice error	aso_agenttype or aso_aistatus missing/misnamed	Open ASO.Core -> Choices and confirm names.
Table already exists	Script rerun or previous partial success	Safe; script skips table creation.
Column already exists	Partial previous run	Safe; script skips existing columns.
No Main method	Program.cs pasted incorrectly	Replace Program.cs with the full script.
Fields not visible	Publishing/cache issue	Refresh maker portal or publish all customizations.

Risks and mitigations

Risk	Mitigation
Wrong environment targeted	Print and verify DataverseUrl before running. Use pac auth list if unsure.
Wrong solution layer	Use SolutionUniqueName = ASOCore and validate in ASO.Core.
Payload fields contain sensitive data	Define retention, masking, and no-secret logging policy before production use.
Global choice mismatch	Verify Step 2 global choices before running script.
Deletion after data exists	Do impact analysis before deleting schema. Prefer backup/recovery over blind deletion.

Rollback / recovery approach

Because this is schema metadata, rollback should be controlled. If the table was created incorrectly and no dependencies or records exist, it can be removed from the unmanaged trial environment after impact review. If records or dependencies exist, do not delete blindly. Use the managed/unmanaged ASO.Core backups, compare with expected schema, and correct the issue through solution-aware changes. In production, rollback should follow formal ALM and change-management processes.

Handover notes for customer IT

- Store Program.cs in source control.
- Record who ran the script, when it ran, and which environment was targeted.
- Keep the backup files named with solution, phase, environment, scope, managed/unmanaged, version, and date.
- Do not store secrets in Input Payload or Output Payload.
- Plan data retention for Agent Run Log before production.
- Create operational views in a later step for failed runs, SAP-called runs, low-confidence runs, and correlation ID investigation.

Appendix A - Full script used

```
using Microsoft.Crm.Sdk.Messages;
using Microsoft.PowerPlatform.Dataverse.Client;
using Microsoft.Xrm.Sdk;
using Microsoft.Xrm.Sdk.Messages;
using Microsoft.Xrm.Sdk.Metadata;

const string DataverseUrl = "https://phoenicarix-ci.crm4.dynamics.com";
const string SolutionUniqueName = "ASOCore";
const string EntityLogicalName = "aso_agentrunlog";
const int LanguageCode = 1033;

var connectionString =
    $@"AuthType=OAuth;
      Url={DataverseUrl};
      LoginPrompt=Auto;
      ClientId=51f81489-12ee-4a9e-aaae-a2591f45987d;
      RedirectUri=http://localhost";

using var service = new ServiceClient(connectionString);

if (!service.IsReady)
{
    Console.ForegroundColor = ConsoleColor.Red;
    Console.WriteLine("Connection failed:");
    Console.WriteLine(service.LastError);
    Console.ResetColor();
    return;
}

Console.WriteLine($"Connected to: {DataverseUrl}");
Console.WriteLine($"Target solution: {SolutionUniqueName}");
Console.WriteLine($"Target table: {EntityLogicalName}");
Console.WriteLine();

if (!EntityExists(service, EntityLogicalName))
{
    Console.WriteLine("Creating Agent Run Log table...");

    var createEntityRequest = new CreateEntityRequest
    {
        Entity = new EntityMetadata
        {
            SchemaName = "aso_agentrunlog",
            DisplayName = Label("Agent Run Log"),
            DisplayCollectionName = Label("Agent Run Logs"),
            Description = Label("Audit, traceability, replay review, and AI/orchestration troubleshooting table for Agentic Sales Orchestrator."),
            OwnershipType = OwnershipTypes.OrganizationOwned
        },
        PrimaryAttribute = new StringAttributeMetadata
```

```
        {
            SchemaName = "aso_name",
            DisplayName = Label("Name"),
            Description = Label("Primary name for the Agent Run Log record."),
            RequiredLevel = new AttributeRequiredLevelManagedProperty(AttributeRequiredLevel.ApplicationRequired),
            MaxLength = 200
        },
        SolutionUniqueName = SolutionUniqueName
    };

    service.Execute(createEntityRequest);

    Console.ForegroundColor = ConsoleColor.Green;
    Console.WriteLine("CREATED TABLE: aso_agentrunlog");
    Console.ResetColor();
}
else
{
    Console.ForegroundColor = ConsoleColor.Yellow;
    Console.WriteLine("SKIP existing table: aso_agentrunlog");
    Console.ResetColor();
}

var existingAttributes = GetExistingAttributeLogicalNames(service, EntityLogicalName);

var fields = new List<AttributeMetadata>
{
    GlobalChoice("aso_agenttype", "Agent Type", "aso_agenttype", "Agent or component type that produced this run log."),
    Text("aso_recordtype", "Record Type", 100, "Logical or business type of the related Dataverse record."),
    Text("aso_recordid", "Record ID", 100, "Identifier of the related Dataverse record."),
    Text("aso_messageid", "Message ID", 100, "Message or event identifier for traceability."),
    Text("aso_correlationid", "Correlation ID", 100, "Correlation ID shared across Power Automate, Foundry, Dataverse, and integration components."),

    Memo("aso_inputpayload", "Input Payload", "Input payload used for the agent/orchestration run."),
    Memo("aso_outputpayload", "Output Payload", "Output payload returned by the agent/orchestration run."),

    DecimalNumber("aso_confidence", "Confidence", 0m, 1m, 2, "Confidence score between 0.00 and 1.00."),
    GlobalChoice("aso_status", "Status", "aso_aistatus", "Processing status for the agent/orchestration run."),
    Memo("aso_errormessage", "Error Message", "Error details if the agent/orchestration run failed."),

    DateTimeField("aso_startedon", "Started On", "Timestamp when the run started."),
    DateTimeField("aso_finishedon", "Finished On", "Timestamp when the run finished."),

    Text("aso_traceid", "Trace ID", 100, "Trace ID for observability and diagnostics."),
    Memo("aso_sourcesused", "Sources Used", "Sources, systems, or context used by the agent/orchestration run."),

    YesNo("aso_sapcalled", "SAP Called", "Indicates whether the run called SAP through the governed integration path."),
    WholeNumber("aso_retrycount", "Retry Count", 0, 100, "Number of retry attempts."),

    Text("aso_foundryrunid", "Foundry Run ID", 100, "Microsoft Foundry run identifier."),
```

```
    Text("aso_salesagentrunid", "Sales Agent Run ID", 100, "Sales Agent run identifier."),
    Text("aso_customerinsightsjourneyid", "Customer Insights Journey ID", 100, "Related Customer Insights journey identifier, if applicable.")
};

foreach (var field in fields)
{
    var logicalName = field.SchemaName!.ToLowerInvariant();

    if (existingAttributes.Contains(logicalName))
    {
        Console.ForegroundColor = ConsoleColor.Yellow;
        Console.WriteLine($"SKIP existing: {logicalName}");
        Console.ResetColor();
        continue;
    }

    try
    {
        Console.WriteLine($"Creating: {logicalName} ...");

        var request = new CreateAttributeRequest
        {
            EntityName = EntityLogicalName,
            Attribute = field,
            SolutionUniqueName = SolutionUniqueName
        };

        service.Execute(request);

        Console.ForegroundColor = ConsoleColor.Green;
        Console.WriteLine($"CREATED: {logicalName}");
        Console.ResetColor();
    }
    catch (Exception ex)
    {
        Console.ForegroundColor = ConsoleColor.Red;
        Console.WriteLine($"FAILED: {logicalName}");
        Console.WriteLine(ex.Message);
        Console.ResetColor();
    }
}

Console.WriteLine();
Console.WriteLine("Publishing Agent Run Log table customizations...");

service.Execute(new PublishXmlRequest
{
    ParameterXml = "<importexportxml><entities><entity>aso_agentrunlog</entity></entities></importexportxml>"
});
```

```
Console.ForegroundColor = ConsoleColor.Green;
Console.WriteLine("Done. Validate in ASO.Core → Tables → Agent Run Log → Columns.");
Console.ResetColor();

static bool EntityExists(ServiceClient service, string entityLogicalName)
{
    try
    {
        service.Execute(new RetrieveEntityRequest
        {
            LogicalName = entityLogicalName,
            EntityFilters = EntityFilters.Entity,
            RetrieveAsIfPublished = true
        });

        return true;
    }
    catch
    {
        return false;
    }
}

static HashSet<string> GetExistingAttributeLogicalNames(ServiceClient service, string entityLogicalName)
{
    var response = (RetrieveEntityResponse)service.Execute(new RetrieveEntityRequest
    {
        LogicalName = entityLogicalName,
        EntityFilters = EntityFilters.Attributes,
        RetrieveAsIfPublished = true
    });

    return response.EntityMetadata.Attributes
        .Where(a => !string.IsNullOrWhiteSpace(a.LogicalName))
        .Select(a => a.LogicalName!)
        .ToHashSet(StringComparer.OrdinalIgnoreCase);
}

static Label Label(string value) => new(value, LanguageCode);

static AttributeRequiredLevelManagedProperty Optional()
{
    return new AttributeRequiredLevelManagedProperty(AttributeRequiredLevel.None);
}

static StringAttributeMetadata Text(string schemaName, string displayName, int maxLength, string description)
{
    return new StringAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
```

```
        Description = Label(description),
        RequiredLevel = Optional(),
        MaxLength = maxLength
    };
}

static MemoAttributeMetadata Memo(string schemaName, string displayName, string description)
{
    return new MemoAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        MaxLength = 4000
    };
}

static DecimalAttributeMetadata DecimalNumber(string schemaName, string displayName, decimal min, decimal max, int precision, string description)
{
    return new DecimalAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        MinValue = min,
        MaxValue = max,
        Precision = precision
    };
}

static IntegerAttributeMetadata WholeNumber(string schemaName, string displayName, int min, int max, string description)
{
    return new IntegerAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        MinValue = min,
        MaxValue = max,
        Format = IntegerFormat.None
    };
}

static DateTimeAttributeMetadata DateTimeField(string schemaName, string displayName, string description)
{
    return new DateTimeAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
```

```
        Description = Label(description),
        RequiredLevel = Optional(),
        Format = DateTimeFormat.DateAndTime,
        DateTimeBehavior = DateTimeBehavior.UserLocal
    };
}

static BooleanAttributeMetadata YesNo(string schemaName, string displayName, string description)
{
    return new BooleanAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        DefaultValue = false,
        OptionSet = new BooleanOptionSetMetadata(
            new OptionMetadata(Label("Yes"), 1),
            new OptionMetadata(Label("No"), 0)
        )
    };
}

static PicklistAttributeMetadata GlobalChoice(string schemaName, string displayName, string globalChoiceName, string description)
{
    return new PicklistAttributeMetadata
    {
        SchemaName = schemaName,
        DisplayName = Label(displayName),
        Description = Label(description),
        RequiredLevel = Optional(),
        OptionSet = new OptionSetMetadata
        {
            IsGlobal = true,
            Name = globalChoiceName,
            OptionSetType = OptionSetType.Picklist
        }
    };
}
```

Appendix B - True line-by-line script explanation

The table below explains every line of the script, including braces and blank lines. It is intentionally detailed so a junior developer can learn from it and a senior consultant can review the pattern.

Line	Code	Explanation in simple language	Technical explanation	Why it matters	Common mistake / warning
1	using Microsoft.Crm.Sdk.Messages;	Imports Dataverse message classes such as PublishXmlRequest.	The script needs this namespace to publish Dataverse customizations after metadata creation.	Without it, PublishXmlRequest may not resolve and the project will not build.	Missing using directives often show as compile errors before the script runs.
2	using Microsoft.PowerPlatform.Dataverse.Client;	Imports ServiceClient, the main	ServiceClient manages OAuth	The script cannot connect to Dataverse	If the NuGet package is missing, this line

		connection object for Dataverse.	authentication and executes SDK requests.	without this package/namespace.	will produce a build error.
3	using Microsoft.Xrm.Sdk;	Imports core SDK types such as Label and Entity-related base types.	Dataverse metadata labels and SDK objects are defined in this namespace.	Many later helper methods use Label and SDK data structures.	If removed, multiple type names will fail to compile.
4	using Microsoft.Xrm.Sdk.Messages;	Imports SDK request and response classes such as CreateEntityRequest and CreateAttributeRequest.	These are the messages that create the table, create columns, and retrieve metadata.	The script is a metadata-deployment script; these request classes are the main engine.	Wrong namespace or missing package stops metadata requests from compiling.
5	using Microsoft.Xrm.Sdk.Metadata;	Imports metadata classes such as EntityMetadata, StringAttributeMetadata, and OptionSetMetadata.	Each Dataverse table/column definition is represented as metadata.	This is how the script describes tables and columns to Dataverse.	Without this import, the column/table metadata classes are unknown.
6	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
7	const string DataverseUrl = "https://phoenicarix-ci.crm4.dynamics.com";	Defines the Dataverse environment URL.	All SDK requests are sent to Phoenicarix-CI.	This prevents accidentally creating the table in the wrong trial environment.	If wrong, the script may create metadata in another environment or fail to connect.
8	const string SolutionUniqueName = "ASOCore";	Defines the target solution unique name.	CreateEntityRequest and CreateAttributeRequest use this value to associate components with ASO.Core.	Solution association is critical for ALM, export, backup, and customer handover.	Using display name instead of unique name can fail or place components outside the expected solution.
9	const string EntityLogicalName = "aso_agentrunlog";	Defines the logical name of the new custom table.	All table checks, column creation, and publishing target aso_agentrunlog.	This is the table that stores AI/orchestration run telemetry.	A typo creates or targets the wrong table name and can break downstream flows.
10	const int LanguageCode = 1033;	Defines the label language code as English.	Dataverse display names and descriptions are localized. 1033 creates English labels.	It makes customer IT documentation and labels predictable.	In a German UI, labels may still appear English unless localized labels are added later.
11	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
12	var connectionString =	Starts building the connection string.	The next lines specify authentication type, URL, client ID, and redirect URI.	ServiceClient needs a connection string to authenticate to Dataverse.	If any part is malformed, ServiceClient will not become ready.
13	$@"AuthType=OAuth;	Starts an interpolated verbatim string and sets OAuth authentication.	OAuth allows interactive sign-in for a local admin/maker running the script.	It avoids storing passwords or secrets in the code.	Do not replace with legacy auth types.
14	Url={DataverseUrl};	Adds the target Dataverse URL into the connection string.	The value comes from the DataverseUrl constant.	Centralizing the URL avoids editing it in several places.	If the URL includes the wrong environment, metadata will go to the wrong tenant/environment.
15	LoginPrompt=Auto;	Allows the SDK to prompt for login when needed.	On macOS this can open a browser or interactive login process.	It keeps the local script usable without service-principal setup for the MVP.	If interactive login is blocked, use an app registration/service principal in a later enterprise version.
16	ClientId=51f81489-12ee-4a9e-aaae-a2591f45987d;	Uses the public Microsoft tooling client ID commonly used for Dataverse interactive tooling.	It identifies the client application for OAuth sign-in.	It is practical for local execution during MVP development.	For production automation, customer IT should review whether a dedicated app registration is required.
17	RedirectUri=http://localhost";	Defines localhost as the OAuth redirect URI.	After browser sign-in, the token flow returns control to the local process.	This is common for local developer tooling.	If blocked by tenant policy, authentication may fail.
18	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
19	using var service = new ServiceClient(connectionString);	Creates the Dataverse service client.	This object executes all later retrieve, create, and publish requests.	It is the main connection object for the script.	If ServiceClient is not ready, no metadata operations should run.
20	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
21	if (!service.IsReady)	Checks whether the connection is usable.	ServiceClient sets IsReady after authentication and connection initialization.	This protects the environment from partially running operations without a valid connection.	Skipping this check makes failures less clear.
22	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
23	Console.ForegroundColor = ConsoleColor.Red;	Changes terminal output to red.	The operator can visually identify failures quickly.	This is operational feedback only; it does not affect Dataverse.	Always reset the color later to avoid confusing output.
24	Console.WriteLine("Connection failed:");	Prints a clear connection failure heading.	It tells the operator that the script stopped before metadata changes.	This supports troubleshooting by separating connection errors from schema errors.	Without clear logging, junior developers may not know where the failure occurred.
25	Console.WriteLine(service.LastError);	Prints the detailed ServiceClient error.	The SDK error usually explains authentication, URL, or permission problems.	This is the first diagnostic signal for support.	Do not suppress this during development.
26	Console.ResetColor();	Resets terminal color to normal.	It prevents later output from staying red, green, or yellow.	Clean console output matters for evidence screenshots and handover.	Forgetting this does not break Dataverse but makes logs confusing.
27	return;	Stops execution immediately.	If connection failed, no schema change should be attempted.	This is a safety guard.	Without it, later code could throw misleading errors.
28	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.
29	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
30	Console.WriteLine($"Connected to: {DataverseUrl}");	Prints the connected environment URL.	The operator can verify the target before changes are created.	This reduces the risk of environment mix-ups.	Always read this line before trusting the run.
31	Console.WriteLine($"Target solution: {SolutionUniqueName}");	Prints the solution unique name.	It confirms ASOCore is the ALM container	This is important for exportable,	If the printed value is wrong, stop and

			being targeted.	customer-ready schema work.	correct the script.
32	Console.WriteLine($"Target table: {EntityLogicalName}");	Prints the target table logical name.	It confirms the script targets aso_agentrunlog.	This avoids accidentally running a table-specific script against another table.	If wrong, stop immediately.
33	Console.WriteLine();	Writes a blank line to the terminal.	It improves readability between log sections.	This helps screenshots and run logs remain understandable.	No technical risk if removed.
34	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
35	if (!EntityExists(service, EntityLogicalName))	Checks whether the Agent Run Log table already exists.	The custom helper sends a RetrieveEntityRequest and returns true/false.	This makes the script safe to rerun and avoids duplicate-table errors.	If missing, duplicate runs fail or may try unnecessary creation.
36	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
37	Console.WriteLine("Creating Agent Run Log table...");	Prints that table creation is starting.	It gives the operator a clear phase marker.	Useful for evidence and troubleshooting.	No technical risk if removed, but logging becomes weaker.
38	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
39	var createEntityRequest = new CreateEntityRequest	Creates an SDK request object for a new Dataverse table.	CreateEntityRequest is the Dataverse metadata operation for table creation.	This is the key line that prepares custom table creation.	Wrong request type would not create a table.
40	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
41	Entity = new EntityMetadata	Starts defining the custom table metadata.	EntityMetadata describes the table schema, labels, description, and ownership.	Dataverse tables are created from metadata definitions.	Incomplete metadata can lead to invalid or poorly named tables.
42	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
43	SchemaName = "aso_agentrunlog",	Sets the schema name of the new table.	The publisher prefix aso_ marks this as an ASO component.	Consistent schema naming supports ALM and maintainability.	Changing after creation is not practical; confirm before running.
44	DisplayName = Label("Agent Run Log"),	Sets the singular display label.	This is what makers and users see for one record/table item.	Readable labels help non-technical stakeholders understand the table.	Poor labels cause confusion in model-driven apps and forms.
45	DisplayCollectionName = Label("Agent Run Logs"),	Sets the plural display label.	This is what makers and app users see for the table collection.	It makes navigation and views readable.	Wrong plural names look unprofessional in the UI.
46	Description = Label("Audit, traceability, replay review, and AI/orchestration troubleshooting table for Agentic Sales Orchestrator."),	Adds a human-readable table description.	Descriptions help future makers understand why the table exists.	This supports customer IT handover and governance.	Leaving descriptions empty makes the schema harder to audit.
47	OwnershipType = OwnershipTypes.OrganizationOwned	Makes the table organization-owned.	Records are not owned by individual users or teams. This fits technical run logs.	Run logs are operational records rather than seller-owned business records.	If user-owned, security and ownership behavior may become more complex.
48	},	Closes one object initializer item and continues the surrounding initializer.	This is used when an item is part of a larger object or list.	It keeps multiple metadata properties/items separated.	Wrong comma placement can stop compilation.
49	PrimaryAttribute = new StringAttributeMetadata	Starts defining the primary name column.	Every custom Dataverse table needs a primary name attribute.	This primary field helps identify records in views and lookups.	Missing primary attribute makes table creation invalid.
50	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
51	SchemaName = "aso_name",	Sets the schema name for the primary name column.	aso_name is the custom primary name attribute for Agent Run Log.	It follows the ASO prefix standard.	Wrong schema names cannot be renamed easily after creation.
52	DisplayName = Label("Name"),	Sets the display name for the primary name column.	Users and makers see this as the primary label.	A simple Name field is sufficient for log records.	Unclear labels make views hard to read.
53	Description = Label("Primary name for the Agent Run Log record."),	Adds a description for the primary name field.	It clarifies that this is the primary identifier field.	Descriptions support maintainability.	No functional risk, but missing descriptions reduce documentation quality.
54	RequiredLevel = new AttributeRequiredLevelManagedProperty(AttributeRequiredLevel.ApplicationRequired),	Makes the primary name application-required.	Dataverse expects a primary name value for custom table records.	This supports readable records in views and lookups.	If not supplied during record creation, future flows may fail unless they set aso_name.
55	MaxLength = 200	Limits the primary name to 200 characters.	StringAttributeMetadata requires sensible text length limits.	This is enough for generated run names or correlation summaries.	Too short clips useful names; too long is unnecessary.
56	},	Closes one object initializer item and continues the surrounding initializer.	This is used when an item is part of a larger object or list.	It keeps multiple metadata properties/items separated.	Wrong comma placement can stop compilation.
57	SolutionUniqueName = SolutionUniqueName	Associates the created table or column with ASO.Core.	The value resolves to ASOCore.	This is critical so the component is exportable with the solution.	If wrong or missing, components may land outside the intended solution.
58	};	Closes an object/list initializer and ends the statement.	This completes metadata definitions such as requests, attribute objects, or lists.	C# needs this syntax to finish the object creation statement.	Missing semicolon or brace causes build errors.
59	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
60	service.Execute(createEntityRequest);	Sends the table creation request to Dataverse.	This is where the table is actually created.	All preceding metadata definitions are applied by this call.	If it fails, read the exception carefully before rerunning.
61	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates	Readable scripts are easier to review,	No technical risk if removed, but the

			code sections for readability.	hand over, and troubleshoot.	script becomes harder to read.
62	Console.ForegroundColor = ConsoleColor.Green;	Changes terminal output to green.	Green indicates success messages.	This helps operators distinguish created items from warnings and errors.	Remember to reset the color later.
63	Console.WriteLine("CREATED TABLE: aso_agentrunlog");	Prints success after table creation.	It confirms that the table creation request succeeded.	This is useful evidence for implementation records.	Do not assume columns are created yet; this only confirms the table.
64	Console.ResetColor();	Resets terminal color to normal.	It prevents later output from staying red, green, or yellow.	Clean console output matters for evidence screenshots and handover.	Forgetting this does not break Dataverse but makes logs confusing.
65	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.
66	else	Executes part of the C# script structure or metadata definition.	This line contributes to creating, configuring, checking, or publishing Dataverse metadata.	It is part of the complete sequence needed for safe programmatic schema creation.	If changed, test by building and running against a sandbox/trial environment first.
67	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
68	Console.ForegroundColor = ConsoleColor.Yellow;	Executes part of the C# script structure or metadata definition.	This line contributes to creating, configuring, checking, or publishing Dataverse metadata.	It is part of the complete sequence needed for safe programmatic schema creation.	If changed, test by building and running against a sandbox/trial environment first.
69	Console.WriteLine("SKIP existing table: aso_agentrunlog");	Prints that the table already exists and will not be recreated.	This supports idempotency and safe reruns.	A skipped table is not a failure if it already exists.	If you expected a new table, verify you are in the right environment.
70	Console.ResetColor();	Resets terminal color to normal.	It prevents later output from staying red, green, or yellow.	Clean console output matters for evidence screenshots and handover.	Forgetting this does not break Dataverse but makes logs confusing.
71	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.
72	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
73	var existingAttributes = GetExistingAttributeLogicalNames(service, EntityLogicalName);	Retrieves the existing column logical names for Agent Run Log.	The helper reads metadata and returns a set of logical names.	This lets the script skip columns already created in a previous or partial run.	Without it, reruns would fail on duplicate columns.
74	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
75	var fields = new List<AttributeMetadata>	Starts the in-memory list of planned column definitions.	Each item is an AttributeMetadata object that describes one Dataverse column.	This list is the script's field inventory.	If a field is omitted here, the script will not create it.
76	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
77	GlobalChoice("aso_agenttype", "Agent Type", "aso_agenttype", "Agent or component type that produced this run log."),	Defines the Agent Type column as a field that reuses an existing global choice.	Creates PicklistAttributeMetadata for aso_agenttype and binds it to a global option set, based on the arguments in this line/block.	Identifies which agent, child agent, journey, or integration component produced the log entry.	The referenced global choice must already exist in ASO.Core, or creation fails.
78	Text("aso_recordtype", "Record Type", 100, "Logical or business type of the related Dataverse record."),	Defines a single-line text column named Record Type.	Calls the Text helper to create StringAttributeMetadata for aso_recordtype.	Stores the related business record type such as lead, opportunity, account, contact, or a custom table name.	Wrong length or schema name can cause truncation or ALM inconsistency.
79	Text("aso_recordid", "Record ID", 100, "Identifier of the related Dataverse record."),	Defines a single-line text column named Record ID.	Calls the Text helper to create StringAttributeMetadata for aso_recordid.	Stores the related record identifier without forcing a polymorphic lookup in this MVP.	Wrong length or schema name can cause truncation or ALM inconsistency.
80	Text("aso_messageid", "Message ID", 100, "Message or event identifier for traceability."),	Defines a single-line text column named Message ID.	Calls the Text helper to create StringAttributeMetadata for aso_messageid.	Stores a message/event identifier for replay and traceability.	Wrong length or schema name can cause truncation or ALM inconsistency.
81	Text("aso_correlationid", "Correlation ID", 100, "Correlation ID shared across Power Automate, Foundry, Dataverse, and integration components."),	Defines a single-line text column named Correlation ID.	Calls the Text helper to create StringAttributeMetadata for aso_correlationid.	Connects Power Automate, Foundry, Dataverse, future APIM/Functions, and monitoring traces.	Wrong length or schema name can cause truncation or ALM inconsistency.
82	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
83	Memo("aso_inputpayload", "Input Payload", "Input payload used for the agent/orchestration run."),	Defines a multiline text column named Input Payload.	Calls the Memo helper to create MemoAttributeMetadata for aso_inputpayload with a 4000-character limit.	Captures the request/context sent into the agent or orchestration run.	Very large payloads may exceed multiline text limits; consider external storage for production-scale logs.
84	Memo("aso_outputpayload", "Output Payload", "Output payload returned by the agent/orchestration run."),	Defines a multiline text column named Output Payload.	Calls the Memo helper to create MemoAttributeMetadata for aso_outputpayload with a 4000-character limit.	Captures the response or decision returned by the agent or orchestration run.	Very large payloads may exceed multiline text limits; consider external storage for production-scale logs.
85	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
86	DecimalNumber("aso_confidence", "Confidence", 0m, 1m, 2, "Confidence score between 0.00 and 1.00."),	Defines the decimal number column Confidence.	Calls DecimalNumber to create DecimalAttributeMetadata for aso_confidence.	Stores a normalized confidence score to drive human review or escalation rules.	Incorrect precision or range makes downstream thresholds unreliable.
87	GlobalChoice("aso_status", "Status", "aso_aistatus", "Processing status for the agent/orchestration run."),	Defines the Status column as a field that reuses an existing global choice.	Creates PicklistAttributeMetadata for aso_status and binds it to a global option set, based on the arguments in this	Tracks execution status using the shared ASO AI status taxonomy.	The referenced global choice must already exist in ASO.Core, or creation fails.

			line/block.		
88	Memo("aso_errormessage", "Error Message", "Error details if the agent/orchestration run failed."),	Defines a multiline text column named Error Message.	Calls the Memo helper to create MemoAttributeMetadata for aso_errormessage with a 4000-character limit.	Stores error details if the run failed.	Very large payloads may exceed multiline text limits; consider external storage for production-scale logs.
89	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
90	DateTimeField("aso_startedon", "Started On", "Timestamp when the run started."),	Defines the date/time column Started On.	Calls DateTimeField to create DateTimeAttributeMetadata for aso_startedon.	Captures when execution started.	Timezone behavior must be understood when reporting across regions.
91	DateTimeField("aso_finishedon", "Finished On", "Timestamp when the run finished."),	Defines the date/time column Finished On.	Calls DateTimeField to create DateTimeAttributeMetadata for aso_finishedon.	Captures when execution completed.	Timezone behavior must be understood when reporting across regions.
92	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
93	Text("aso_traceid", "Trace ID", 100, "Trace ID for observability and diagnostics."),	Defines a single-line text column named Trace ID.	Calls the Text helper to create StringAttributeMetadata for aso_traceid.	Stores observability trace references for diagnostics.	Wrong length or schema name can cause truncation or ALM inconsistency.
94	Memo("aso_sourcesused", "Sources Used", "Sources, systems, or context used by the agent/orchestration run."),	Defines a multiline text column named Sources Used.	Calls the Memo helper to create MemoAttributeMetadata for aso_sourcesused with a 4000-character limit.	Stores sources, systems, and context used by the run.	Very large payloads may exceed multiline text limits; consider external storage for production-scale logs.
95	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
96	YesNo("aso_sapcalled", "SAP Called", "Indicates whether the run called SAP through the governed integration path."),	Defines the Yes/No column SAP Called.	Calls YesNo to create BooleanAttributeMetadata for aso_sapcalled.	Indicates whether SAP was called through the governed path.	Default value is false; future automation must set true when applicable.
97	WholeNumber("aso_retrycount", "Retry Count", 0, 100, "Number of retry attempts."),	Defines the whole number column Retry Count.	Calls WholeNumber to create IntegerAttributeMetadata for aso_retrycount.	Stores retry attempts for operational diagnosis.	Incorrect min/max values can block valid data or allow invalid values.
98	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
99	Text("aso_foundryrunid", "Foundry Run ID", 100, "Microsoft Foundry run identifier."),	Defines a single-line text column named Foundry Run ID.	Calls the Text helper to create StringAttributeMetadata for aso_foundryrunid.	Stores Microsoft Foundry run identifier.	Wrong length or schema name can cause truncation or ALM inconsistency.
100	Text("aso_salesagentrunid", "Sales Agent Run ID", 100, "Sales Agent run identifier."),	Defines a single-line text column named Sales Agent Run ID.	Calls the Text helper to create StringAttributeMetadata for aso_salesagentrunid.	Stores Dynamics Sales Agent/Copilot run identifier.	Wrong length or schema name can cause truncation or ALM inconsistency.
101	Text("aso_customerinsightsjourneyid", "Customer Insights Journey ID", 100, "Related Customer Insights journey identifier, if applicable.")	Defines a single-line text column named Customer Insights Journey ID.	Calls the Text helper to create StringAttributeMetadata for aso_customerinsightsjourneyid.	Stores related Customer Insights journey identifier if the log belongs to a journey action.	Wrong length or schema name can cause truncation or ALM inconsistency.
102	};	Closes an object/list initializer and ends the statement.	This completes metadata definitions such as requests, attribute objects, or lists.	C# needs this syntax to finish the object creation statement.	Missing semicolon or brace causes build errors.
103	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
104	foreach (var field in fields)	Starts a loop over every planned field.	The script processes one field definition at a time.	This is how many columns can be created from one reusable script.	If the list is empty, no columns will be created.
105	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
106	var logicalName = field.SchemaName!.ToLowerInvariant();	Reads the schema name and normalizes it to lowercase.	Dataverse logical names are lowercase; the comparison should be case-insensitive and consistent.	This makes duplicate checks reliable.	If schema name is null or wrong, this line can fail or compare the wrong value.
107	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
108	if (existingAttributes.Contains(logicalName))	Checks whether the field already exists.	existingAttributes is a set returned from Dataverse metadata.	This prevents duplicate column creation errors.	Without it, a rerun would fail on the first existing column.
109	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
110	Console.ForegroundColor = ConsoleColor.Yellow;	Executes part of the C# script structure or metadata definition.	This line contributes to creating, configuring, checking, or publishing Dataverse metadata.	It is part of the complete sequence needed for safe programmatic schema creation.	If changed, test by building and running against a sandbox/trial environment first.
111	Console.WriteLine($"SKIP existing: {logicalName}");	Prints that an existing field was skipped.	The field is not recreated because it is already in Dataverse.	This gives safe rerun evidence.	If too many are skipped unexpectedly, check whether the script was already run.
112	Console.ResetColor();	Resets terminal color to normal.	It prevents later output from staying red, green, or yellow.	Clean console output matters for evidence screenshots and handover.	Forgetting this does not break Dataverse but makes logs confusing.
113	continue;	Moves to the next field in the loop.	It avoids running the create request for an existing field.	This is part of idempotent behavior.	Without continue, the script would still try to create the duplicate field.
114	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.

115	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
116	try	Starts a protected block for field creation.	If creation fails for one field, the catch block can print a clear error.	This gives field-level troubleshooting rather than a vague crash.	Do not hide exceptions silently.
117	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
118	Console.WriteLine($"Creating: {logicalName} ...");	Prints the field currently being created.	This produces a live execution log.	It helps connect errors to exact fields.	Without it, troubleshooting is harder.
119	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
120	var request = new CreateAttributeRequest	Creates an SDK request to add one column to the table.	CreateAttributeRequest is the Dataverse metadata message for column creation.	This is the main column creation operation.	Wrong request type or metadata causes field creation failures.
121	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
122	EntityName = EntityLogicalName,	Sets the target table for the new column.	For this script it resolves to aso_agentrunlog.	Ensures the column is added to Agent Run Log.	Wrong entity name creates the field on the wrong table or fails.
123	Attribute = field,	Supplies the current field metadata to the create request.	The metadata object defines the column type, schema name, labels, and settings.	This is how the loop turns each field definition into an actual Dataverse column.	If field metadata is invalid, this request will fail.
124	SolutionUniqueName = SolutionUniqueName	Associates the created table or column with ASO.Core.	The value resolves to ASOCore.	This is critical so the component is exportable with the solution.	If wrong or missing, components may land outside the intended solution.
125	};	Closes an object/list initializer and ends the statement.	This completes metadata definitions such as requests, attribute objects, or lists.	C# needs this syntax to finish the object creation statement.	Missing semicolon or brace causes build errors.
126	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
127	service.Execute(request);	Sends the create-column request to Dataverse.	The SDK executes the metadata operation against the connected environment.	This is the line that actually creates the field.	Permissions, invalid choice names, or duplicate schema names can fail here.
128	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
129	Console.ForegroundColor = ConsoleColor.Green;	Changes terminal output to green.	Green indicates success messages.	This helps operators distinguish created items from warnings and errors.	Remember to reset the color later.
130	Console.WriteLine($"CREATED: {logicalName}");	Prints successful column creation.	The green terminal output confirms each field created correctly.	Useful for screenshots and implementation evidence.	Still validate in the maker portal after the run.
131	Console.ResetColor();	Resets terminal color to normal.	It prevents later output from staying red, green, or yellow.	Clean console output matters for evidence screenshots and handover.	Forgetting this does not break Dataverse but makes logs confusing.
132	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.
133	catch (Exception ex)	Starts error handling for a failed field creation attempt.	Any exception thrown in the try block is captured as ex.	The script can print the specific Dataverse or SDK error message.	Do not ignore errors; a red FAILED line means validation is required.
134	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
135	Console.ForegroundColor = ConsoleColor.Red;	Changes terminal output to red.	The operator can visually identify failures quickly.	This is operational feedback only; it does not affect Dataverse.	Always reset the color later to avoid confusing output.
136	Console.WriteLine($"FAILED: {logicalName}");	Prints the field that failed.	This identifies the exact column that needs investigation.	Useful when one field fails but others succeed.	Do not continue to later phases without reviewing failures.
137	Console.WriteLine(ex.Message);	Prints the technical exception message.	Dataverse often explains missing global choices, invalid schema names, or permission problems here.	This is the main troubleshooting clue.	A screenshot of this line helps support review.
138	Console.ResetColor();	Resets terminal color to normal.	It prevents later output from staying red, green, or yellow.	Clean console output matters for evidence screenshots and handover.	Forgetting this does not break Dataverse but makes logs confusing.
139	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.
140	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.
141	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
142	Console.WriteLine();	Writes a blank line to the terminal.	It improves readability between log sections.	This helps screenshots and run logs remain understandable.	No technical risk if removed.
143	Console.WriteLine("Publishing Agent Run Log table customizations...");	Prints that publishing is starting.	Metadata changes must be published to become usable in the maker/runtime experience.	This is the transition from creation to availability.	If publish fails, fields may exist but not be fully usable.
144	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
145	service.Execute(new PublishXmlRequest	Starts a targeted publish request.	PublishXmlRequest publishes the specified table metadata.	This avoids publishing everything in the environment.	Wrong publish XML may not publish the intended table.
146	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop,	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.

			or try/catch.		
147	ParameterXml = "<importexportxml><entities><entity>aso_agentrunlog</entity></entities></importexportxml>"	Specifies that only aso_agentrunlog should be published.	The XML format tells Dataverse which entity's customizations to publish.	Targeted publishing is safer and faster than publish-all.	If the entity name is wrong, changes may not become visible.
148	});	Executes part of the C# script structure or metadata definition.	This line contributes to creating, configuring, checking, or publishing Dataverse metadata.	It is part of the complete sequence needed for safe programmatic schema creation.	If changed, test by building and running against a sandbox/trial environment first.
149	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
150	Console.ForegroundColor = ConsoleColor.Green;	Changes terminal output to green.	Green indicates success messages.	This helps operators distinguish created items from warnings and errors.	Remember to reset the color later.
151	Console.WriteLine("Done. Validate in ASO.Core → Tables → Agent Run Log → Columns.");	Prints final completion and validation instruction.	The operator still needs to check the maker portal after successful script execution.	This supports handover discipline.	Never rely only on terminal output; validate in Power Apps.
152	Console.ResetColor();	Resets terminal color to normal.	It prevents later output from staying red, green, or yellow.	Clean console output matters for evidence screenshots and handover.	Forgetting this does not break Dataverse but makes logs confusing.
153	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
154	static bool EntityExists(ServiceClient service, string entityLogicalName)	Declares a helper method that checks whether a table exists.	It uses a RetrieveEntityRequest and returns true or false.	This avoids trying to create the table twice.	If implemented incorrectly, it may hide real connection/permission errors.
155	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
156	try	Starts a protected block for field creation.	If creation fails for one field, the catch block can print a clear error.	This gives field-level troubleshooting rather than a vague crash.	Do not hide exceptions silently.
157	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
158	service.Execute(new RetrieveEntityRequest	Creates a metadata retrieve request.	RetrieveEntityRequest reads table metadata from Dataverse.	The script uses this to check table existence and existing fields.	Wrong entity filters may return too much, too little, or fail.
159	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
160	LogicalName = entityLogicalName,	Sets the logical table name on a retrieve request.	The method receives the target table name as a parameter.	Reusable helper methods work for any table passed in.	Wrong logical name causes retrieve failure.
161	EntityFilters = EntityFilters.Entity,	Requests only table-level metadata.	This is enough for checking whether a table exists.	Keeps the existence check light and focused.	If attributes are needed, use EntityFilters.Attributes instead.
162	RetrieveAsIfPublished = true	Reads metadata including unpublished changes.	Dataverse can have unpublished customizations. This asks for metadata as if published.	Prevents duplicate creation during a session where changes were created but not yet fully published.	Without it, the script might not see recent unpublished metadata.
163	});	Executes part of the C# script structure or metadata definition.	This line contributes to creating, configuring, checking, or publishing Dataverse metadata.	It is part of the complete sequence needed for safe programmatic schema creation.	If changed, test by building and running against a sandbox/trial environment first.
164	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
165	return true;	Returns true from the table-existence helper.	If RetrieveEntityRequest succeeds, the table exists.	This tells the main script to skip table creation.	If placed incorrectly, the helper could return true for failures.
166	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.
167	catch	Starts a catch block without an exception variable.	Used in EntityExists where any retrieve failure is treated as non-existence.	It keeps the existence helper simple for MVP scripting.	Could hide permission errors; production-grade code may inspect exception details.
168	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
169	return false;	Returns false from the table-existence helper.	If the retrieve request throws, the script assumes the table does not exist.	This allows the create-table branch to run.	In production, customer IT may want to distinguish not-found from permission errors.
170	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.
171	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.
172	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
173	static HashSet<string> GetExistingAttributeLogicalNames(ServiceClient service, string entityLogicalName)	Declares a helper method to retrieve existing column names.	It returns a HashSet for fast and case-insensitive existence checks.	This enables safe reruns after partial completion.	If it fails, duplicate checks cannot work.
174	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.

175	var response = (RetrieveEntityResponse)service.Execute(new RetrieveEntityRequest	Creates a metadata retrieve request.	RetrieveEntityRequest reads table metadata from Dataverse.	The script uses this to check table existence and existing fields.	Wrong entity filters may return too much, too little, or fail.
176	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
177	LogicalName = entityLogicalName,	Sets the logical table name on a retrieve request.	The method receives the target table name as a parameter.	Reusable helper methods work for any table passed in.	Wrong logical name causes retrieve failure.
178	EntityFilters = EntityFilters.Attributes,	Requests column/attribute metadata.	The helper needs existing attribute logical names for duplicate checks.	This supports idempotent reruns.	If wrong, existing column detection may fail.
179	RetrieveAsIfPublished = true	Reads metadata including unpublished changes.	Dataverse can have unpublished customizations. This asks for metadata as if published.	Prevents duplicate creation during a session where changes were created but not yet fully published.	Without it, the script might not see recent unpublished metadata.
180	});	Executes part of the C# script structure or metadata definition.	This line contributes to creating, configuring, checking, or publishing Dataverse metadata.	It is part of the complete sequence needed for safe programmatic schema creation.	If changed, test by building and running against a sandbox/trial environment first.
181	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
182	return response.EntityMetadata.Attributes	Starts returning existing attribute names from retrieved metadata.	The following LINQ chain filters and converts metadata into a HashSet.	This supports safe reruns.	If response is null, earlier retrieve request failed.
183	.Where(a => !string.IsNullOrWhiteSpace(a.LogicalName))	Filters attributes to only those with a populated logical name.	LINQ removes null or blank names before comparison.	Protects the HashSet from invalid values.	If removed, null values may cause errors later.
184	.Select(a => a.LogicalName!)	Selects each attribute logical name.	LINQ projects AttributeMetadata objects into strings.	The duplicate check only needs logical names.	Selecting the wrong property would break duplicate detection.
185	.ToHashSet(StringComparer.OrdinalIgnoreCase);	Creates a case-insensitive HashSet of logical names.	StringComparer.OrdinalIgnoreCase makes comparison reliable.	Fast lookup is useful when many fields are checked.	Without case-insensitive comparison, casing differences could cause false duplicates.
186	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.
187	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
188	static Label Label(string value) => new(value, LanguageCode);	Defines a shortcut for creating Dataverse labels.	It applies the configured language code to display names and descriptions.	This keeps metadata creation code concise and consistent.	Wrong language code creates labels in the wrong locale.
189	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
190	static AttributeRequiredLevelManagedProperty Optional()	Declares a helper for optional fields.	It returns AttributeRequiredLevel.None.	Most run log fields should be optional because different run types may populate different fields.	Making too many fields required can break future integrations.
191	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
192	return new AttributeRequiredLevelManagedProperty(AttributeRequiredLevel.None);	Returns an optional required-level setting.	AttributeRequiredLevel.None means the column is not mandatory.	Run log records may not populate every column for every run type.	Making fields required too early can break future automation.
193	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.
194	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
195	static StringAttributeMetadata Text(string schemaName, string displayName, int maxLength, string description)	Defines a helper for single-line text fields.	It creates StringAttributeMetadata with schema name, display label, description, and max length.	Avoids repeating verbose metadata syntax for every text field.	Wrong max length can truncate important identifiers.
196	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
197	return new StringAttributeMetadata	Creates a new text metadata object.	StringAttributeMetadata represents a single-line text column.	Used for IDs, references, record type, correlation IDs, and trace IDs.	Wrong helper choice can create the wrong Dataverse type.
198	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
199	SchemaName = schemaName,	Assigns the schema name passed into the helper.	The helper reuses this property for many field types.	This guarantees each field is created with the intended aso_ logical name.	Schema names cannot be easily changed after creation.
200	DisplayName = Label(displayName),	Assigns the display label passed into the helper.	The Label helper applies the configured language code.	Readable display names help users and makers understand the field.	Wrong labels can be fixed later but create confusion.
201	Description = Label(description),	Assigns the field description.	The Label helper stores it as localized metadata.	Descriptions are important for IT handover and data dictionary quality.	Missing descriptions make future maintenance harder.
202	RequiredLevel = Optional(),	Marks the field as optional.	The Optional helper returns AttributeRequiredLevel.None.	Run log fields vary by agent type and should not all be mandatory.	Overly strict required fields can break integrations.
203	MaxLength = maxLength	Applies the maximum text length passed into the Text helper.	Dataverse needs max length for string attributes.	Controls storage and UI constraints for text fields.	Too-short lengths can truncate important IDs.
204	};	Closes an object/list initializer and ends the statement.	This completes metadata definitions such as requests, attribute objects, or lists.	C# needs this syntax to finish the object creation statement.	Missing semicolon or brace causes build errors.

205	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.
206	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
207	static MemoAttributeMetadata Memo(string schemaName, string displayName, string description)	Defines a helper for multiline text fields.	It creates MemoAttributeMetadata with 4000 character length.	Payloads, errors, and sources need more space than a text field.	Dataverse multiline text has limits; very large payloads may require file/blob logging later.
208	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
209	return new MemoAttributeMetadata	Creates a new multiline text metadata object.	MemoAttributeMetadata represents long text fields.	Used for payloads, error messages, and sources used.	Do not store secrets or unbounded payloads here.
210	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
211	SchemaName = schemaName,	Assigns the schema name passed into the helper.	The helper reuses this property for many field types.	This guarantees each field is created with the intended aso_ logical name.	Schema names cannot be easily changed after creation.
212	DisplayName = Label(displayName),	Assigns the display label passed into the helper.	The Label helper applies the configured language code.	Readable display names help users and makers understand the field.	Wrong labels can be fixed later but create confusion.
213	Description = Label(description),	Assigns the field description.	The Label helper stores it as localized metadata.	Descriptions are important for IT handover and data dictionary quality.	Missing descriptions make future maintenance harder.
214	RequiredLevel = Optional(),	Marks the field as optional.	The Optional helper returns AttributeRequiredLevel.None.	Run log fields vary by agent type and should not all be mandatory.	Overly strict required fields can break integrations.
215	MaxLength = 4000	Sets multiline text maximum length to 4000 characters.	This is adequate for MVP payload/error summaries.	Prevents very long text from growing without limit inside Dataverse.	Large raw payloads should move to external storage later.
216	};	Closes an object/list initializer and ends the statement.	This completes metadata definitions such as requests, attribute objects, or lists.	C# needs this syntax to finish the object creation statement.	Missing semicolon or brace causes build errors.
217	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.
218	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
219	static DecimalAttributeMetadata DecimalNumber(string schemaName, string displayName, decimal min, decimal max, int precision, string description)	Defines a helper for decimal fields.	It supports normalized values such as confidence 0.00-1.00.	Precision 2 is sufficient for confidence display and rules.	Wrong min/max or precision can make downstream thresholds inconsistent.
220	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
221	return new DecimalAttributeMetadata	Creates a new decimal metadata object.	DecimalAttributeMetadata represents decimal numbers.	Used for confidence values.	Use decimal carefully for scores/ranges.
222	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
223	SchemaName = schemaName,	Assigns the schema name passed into the helper.	The helper reuses this property for many field types.	This guarantees each field is created with the intended aso_ logical name.	Schema names cannot be easily changed after creation.
224	DisplayName = Label(displayName),	Assigns the display label passed into the helper.	The Label helper applies the configured language code.	Readable display names help users and makers understand the field.	Wrong labels can be fixed later but create confusion.
225	Description = Label(description),	Assigns the field description.	The Label helper stores it as localized metadata.	Descriptions are important for IT handover and data dictionary quality.	Missing descriptions make future maintenance harder.
226	RequiredLevel = Optional(),	Marks the field as optional.	The Optional helper returns AttributeRequiredLevel.None.	Run log fields vary by agent type and should not all be mandatory.	Overly strict required fields can break integrations.
227	MinValue = min,	Sets the minimum numeric value from the helper argument.	Used for decimal and whole number helpers.	Keeps values within a valid business range.	Wrong min values can reject valid data.
228	MaxValue = max,	Sets the maximum numeric value from the helper argument.	Used for confidence and retry count.	Constrains values for quality and reporting.	Wrong max values can block legitimate data.
229	Precision = precision	Sets decimal precision.	For confidence, precision 2 stores values like 0.75.	Consistent precision supports thresholding and reporting.	Too little precision may lose useful detail.
230	};	Closes an object/list initializer and ends the statement.	This completes metadata definitions such as requests, attribute objects, or lists.	C# needs this syntax to finish the object creation statement.	Missing semicolon or brace causes build errors.
231	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.
232	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
233	static IntegerAttributeMetadata WholeNumber(string schemaName, string displayName, int min, int max, string description)	Defines a helper for whole number fields.	It is used for Retry Count with range 0-100.	Retry counts should be integers, not text or decimals.	Wrong ranges can reject valid retry counts or allow unreasonable values.
234	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
235	return new IntegerAttributeMetadata	Creates a new whole-number metadata	IntegerAttributeMetadata represents	Used for retry count.	If retry can exceed 100, adjust MaxValue.

		object.	integer fields.		
236	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
237	SchemaName = schemaName,	Assigns the schema name passed into the helper.	The helper reuses this property for many field types.	This guarantees each field is created with the intended aso_ logical name.	Schema names cannot be easily changed after creation.
238	DisplayName = Label(displayName),	Assigns the display label passed into the helper.	The Label helper applies the configured language code.	Readable display names help users and makers understand the field.	Wrong labels can be fixed later but create confusion.
239	Description = Label(description),	Assigns the field description.	The Label helper stores it as localized metadata.	Descriptions are important for IT handover and data dictionary quality.	Missing descriptions make future maintenance harder.
240	RequiredLevel = Optional(),	Marks the field as optional.	The Optional helper returns AttributeRequiredLevel.None.	Run log fields vary by agent type and should not all be mandatory.	Overly strict required fields can break integrations.
241	MinValue = min,	Sets the minimum numeric value from the helper argument.	Used for decimal and whole number helpers.	Keeps values within a valid business range.	Wrong min values can reject valid data.
242	MaxValue = max,	Sets the maximum numeric value from the helper argument.	Used for confidence and retry count.	Constrains values for quality and reporting.	Wrong max values can block legitimate data.
243	Format = IntegerFormat.None	Sets the integer format as a plain number.	Retry count is not a duration, language, or timezone field.	This keeps the whole number generic.	Wrong integer format can change UI behavior.
244	};	Closes an object/list initializer and ends the statement.	This completes metadata definitions such as requests, attribute objects, or lists.	C# needs this syntax to finish the object creation statement.	Missing semicolon or brace causes build errors.
245	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.
246	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
247	static DateTimeAttributeMetadata DateTimeField(string schemaName, string displayName, string description)	Defines a helper for date/time fields.	It creates user-local Date and Time columns.	Started/Finished timestamps help calculate durations and troubleshoot stale runs.	Be clear about timezone behavior in enterprise reporting.
248	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
249	return new DateTimeAttributeMetadata	Creates a new date/time metadata object.	DateTimeAttributeMetadata represents Dataverse date/time fields.	Used for run start and finish timestamps.	Choose DateTimeBehavior deliberately for reporting.
250	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
251	SchemaName = schemaName,	Assigns the schema name passed into the helper.	The helper reuses this property for many field types.	This guarantees each field is created with the intended aso_ logical name.	Schema names cannot be easily changed after creation.
252	DisplayName = Label(displayName),	Assigns the display label passed into the helper.	The Label helper applies the configured language code.	Readable display names help users and makers understand the field.	Wrong labels can be fixed later but create confusion.
253	Description = Label(description),	Assigns the field description.	The Label helper stores it as localized metadata.	Descriptions are important for IT handover and data dictionary quality.	Missing descriptions make future maintenance harder.
254	RequiredLevel = Optional(),	Marks the field as optional.	The Optional helper returns AttributeRequiredLevel.None.	Run log fields vary by agent type and should not all be mandatory.	Overly strict required fields can break integrations.
255	Format = DateTimeFormat.DateAndTime,	Sets date/time format to include date and time.	Run timestamps need both date and exact time.	This supports operational troubleshooting.	Date-only would lose important execution timing.
256	DateTimeBehavior = DateTimeBehavior.UserLocal	Sets date/time behavior to user local.	Users see timestamps adjusted to their local timezone.	This is common for business records in Dataverse.	For integration logs, UTC may be preferred in production; confirm later.
257	};	Closes an object/list initializer and ends the statement.	This completes metadata definitions such as requests, attribute objects, or lists.	C# needs this syntax to finish the object creation statement.	Missing semicolon or brace causes build errors.
258	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.
259	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
260	static BooleanAttributeMetadata YesNo(string schemaName, string displayName, string description)	Defines a helper for Yes/No fields.	It creates a BooleanAttributeMetadata with Yes and No labels and default false.	Used for SAP Called because it is a binary operational flag.	Default false is sensible, but future flows must update it when SAP is called.
261	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
262	return new BooleanAttributeMetadata	Creates a new Yes/No metadata object.	BooleanAttributeMetadata represents Dataverse boolean fields.	Used for SAP Called.	Default false must match business semantics.
263	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
264	SchemaName = schemaName,	Assigns the schema name passed into the helper.	The helper reuses this property for many field types.	This guarantees each field is created with the intended aso_ logical name.	Schema names cannot be easily changed after creation.
265	DisplayName = Label(displayName),	Assigns the display label passed into the helper.	The Label helper applies the configured language code.	Readable display names help users and makers understand the field.	Wrong labels can be fixed later but create confusion.
266	Description = Label(description),	Assigns the field description.	The Label helper stores it as localized metadata.	Descriptions are important for IT handover and data dictionary quality.	Missing descriptions make future maintenance harder.

267	RequiredLevel = Optional(),	Marks the field as optional.	The Optional helper returns AttributeRequiredLevel.None.	Run log fields vary by agent type and should not all be mandatory.	Overly strict required fields can break integrations.
268	DefaultValue = false,	Sets the default boolean value to false.	SAP Called should default to No unless a governed SAP call actually occurred.	This avoids false positives.	Automation must set it true when SAP is called.
269	OptionSet = new BooleanOptionSetMetadata(	Starts defining labels for the Yes/No option set.	Boolean fields require true and false option labels.	This produces user-friendly values in Dataverse.	Incorrect labels confuse users but do not change boolean mechanics.
270	new OptionMetadata(Label("Yes"), 1),	Defines the Yes option for a Yes/No field.	Creates the true option with label Yes and value 1.	Dataverse Boolean fields need labels for both states.	Changing values can confuse integrations expecting standard boolean mapping.
271	new OptionMetadata(Label("No"), 0)	Defines the No option for a Yes/No field.	Creates the false option with label No and value 0.	Default false/no is used for fields such as SAP Called.	Do not invert these values unless every integration is adjusted.
272	)	Executes part of the C# script structure or metadata definition.	This line contributes to creating, configuring, checking, or publishing Dataverse metadata.	It is part of the complete sequence needed for safe programmatic schema creation.	If changed, test by building and running against a sandbox/trial environment first.
273	};	Closes an object/list initializer and ends the statement.	This completes metadata definitions such as requests, attribute objects, or lists.	C# needs this syntax to finish the object creation statement.	Missing semicolon or brace causes build errors.
274	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.
275	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.
276	static PicklistAttributeMetadata GlobalChoice(string schemaName, string displayName, string globalChoiceName, string description)	Defines a helper for fields that reuse existing global choices.	It creates a PicklistAttributeMetadata referencing an OptionSetMetadata with IsGlobal=true.	This keeps Agent Type and Status aligned with centrally governed ASO choices.	If the global choice name is wrong, field creation fails.
277	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
278	return new PicklistAttributeMetadata	Creates a new choice metadata object.	PicklistAttributeMetadata represents a single-select choice field.	Used to bind columns to global choices.	Wrong global choice reference causes metadata errors.
279	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
280	SchemaName = schemaName,	Assigns the schema name passed into the helper.	The helper reuses this property for many field types.	This guarantees each field is created with the intended aso_ logical name.	Schema names cannot be easily changed after creation.
281	DisplayName = Label(displayName),	Assigns the display label passed into the helper.	The Label helper applies the configured language code.	Readable display names help users and makers understand the field.	Wrong labels can be fixed later but create confusion.
282	Description = Label(description),	Assigns the field description.	The Label helper stores it as localized metadata.	Descriptions are important for IT handover and data dictionary quality.	Missing descriptions make future maintenance harder.
283	RequiredLevel = Optional(),	Marks the field as optional.	The Optional helper returns AttributeRequiredLevel.None.	Run log fields vary by agent type and should not all be mandatory.	Overly strict required fields can break integrations.
284	OptionSet = new OptionSetMetadata	Executes part of the C# script structure or metadata definition.	This line contributes to creating, configuring, checking, or publishing Dataverse metadata.	It is part of the complete sequence needed for safe programmatic schema creation.	If changed, test by building and running against a sandbox/trial environment first.
285	{	Opens a code block.	The following lines are grouped under a condition, object initializer, method, loop, or try/catch.	Braces define structure and scope in C#.	A missing opening brace causes compile errors or wrong logic scope.
286	IsGlobal = true,	Marks the option set reference as global.	The field will use an existing reusable choice instead of a local option list.	This is important for shared taxonomies such as Agent Type and Status.	If false, a local choice would be created instead, breaking consistency.
287	Name = globalChoiceName,	Provides the logical name of the existing global choice.	Dataverse uses this name to bind the field to the correct global option set.	This is how aso_status binds to aso_aistatus and aso_agenttype binds to aso_agenttype.	Incorrect names cause a runtime metadata error.
288	OptionSetType = OptionSetType.Picklist	Sets the option set type to single-select picklist.	The fields allow one selected value at a time.	This matches the intended run log status and agent type behavior.	Use multi-select only if the business process requires multiple simultaneous values.
289	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.
290	};	Closes an object/list initializer and ends the statement.	This completes metadata definitions such as requests, attribute objects, or lists.	C# needs this syntax to finish the object creation statement.	Missing semicolon or brace causes build errors.
291	}	Closes a code block.	This ends the current condition, method, loop, helper, or object initializer scope.	Correct closing braces keep C# structure valid.	A missing or extra brace is one of the most common paste errors.
292	(blank)	Blank line used for spacing.	It has no runtime behavior; it separates code sections for readability.	Readable scripts are easier to review, hand over, and troubleshoot.	No technical risk if removed, but the script becomes harder to read.